Toronto Metropolitan University

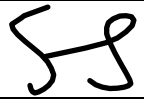
Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Digital Systems
Course Number:	COE328
Semester/Year (e.g.F2016)	F2025

Instructor:	Sedaghat Reza

Assignment/Lab Number:	6
Assignment/Lab Title:	Simple General Purpose Processor

Submission Date:	2025/12/01
Due Date:	2025/12/01

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Juma	Sarah	501316925	16	

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf

Table of Contents

Cover Page …… 1

Table of Contents …… 2

Introduction …… 3

Components (Latch1, Latch2, 4:16 Decoder, FSM) …… 4

Problem Set 1 …… 8

Problem Set 2 …… 11

Problem Set 3 …… 13

Conclusion …… 16

Introduction

This lab is centered around building a simple GPU processor by combining storage elements, a control unit, and ALU core, and a seven-segment display system. The latches act as temporary storage for the 8-bit inputs, A and B, and they are responsible for keeping the values stable so the ALU can correctly process them. The control unit coordinates the whole design by using the FSM from Lab 5 to cycle through states and a 4:16 decoder to convert those states into microcode signals that are delivered to the ALU. These indicate to the ALU which operations to perform as per specific problem sets. The 4:16 decoder uses the FSM outputs to activate exactly one of sixteen possible microcode lines, and it is made from 2 3:8 decoders which follows the two-decoder structure provided. When S3 = 0, the upper decoder is disabled and the lower one generates OP[0..7]. When S3 = 1, the second decoder generates OP[8..15]. The ALU core receives the value of A, B, and the microcode and performs arithmetic or logic operations such as addition, subtraction, inversion, and bitwise logic as per the ALU operation table provided. The seven-segment display unit translates each 40bit value into the corresponding active-low LED pattern, which allows for the full 8-bit ALU result to show on the FPGA Alterra board in hexadecimal format. These components put together form a simple general-purpose processor where the FSM selects an operation, the decoder generates control signals, the latches hold inputs, the ALU computes results, and the display shows the final output.

Components

Latch 1 and Latch 2

Latch 1 and Latch 2 are 8-bit storage units that hold the inputs A and B on the rising edge of the clock, maintaining stable data for the ALU as shown in the truth table below. The ALU could give unstable and incorrect outputs without the use of latches.

Reset	Clock	Input A/B [7:0]	Next Output Q[7:0]	Description
0	X	X	00000000	Reset active → output cleared
1	↑ (rising edge)	00000000	00000000	Captures input on rising edge
1	↑ (rising edge)	01010101	01010101	Captures new input
1	↑ (rising edge)	11111111	11111111	Captures new input
1	0 or ↓	X	Q (unchanged)	No rising edge → holds previous value

Table 1: Truth Table For Components Latch 1 (Input A) and Latch 2 (Input B).

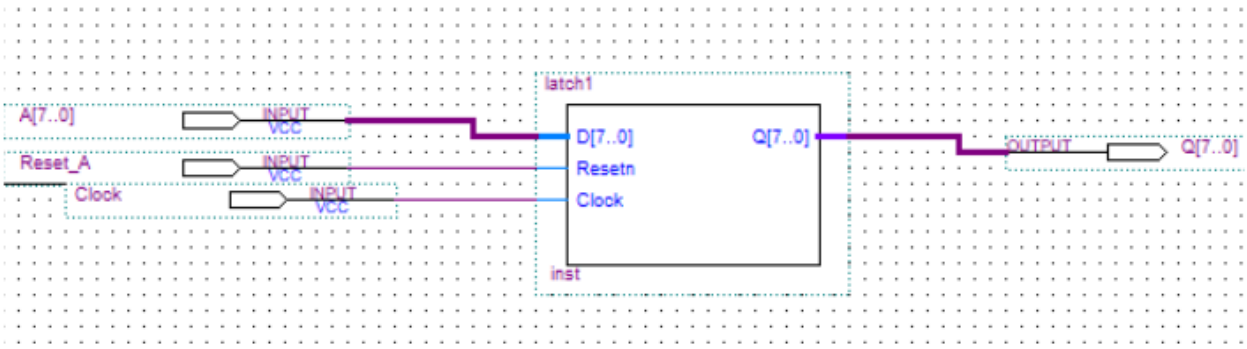


Figure 1: Block Diagram For Latch 1 and Latch 2

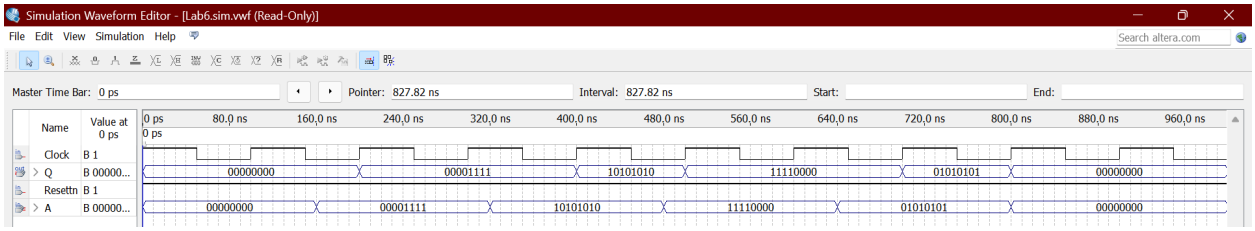


Figure 2: Waveform Output For Latch 1 and Latch 2

4:16 Decoder

The 4:16 decoder takes the 4-bit current state from the FSM to activate one of the 16 output lines, producing the microcode used by the ALU. For this lab, the decoder is built using two 3:8 decoders along with a NOT gate and two AND gates. These select which decoder is enabled based on the most significant input bit.

En	S3	S2	S1	S0	Active Output	OP[15..0]
0	X	X	X	X	None	0000000000000000
1	0	0	0	0	OP0	000000000000 0001
1	0	0	0	1	OP1	0000 0000 0000 0010
1	0	0	1	0	OP2	0000 0000 0000 0100
1	0	0	1	1	OP3	0000 0000 0000 1000
1	0	1	0	0	OP4	0000 0000 0001 0000
1	0	1	0	1	OP5	0000 0000 0010 0000
1	0	1	1	0	OP6	0000 0000 0100 0000
1	0	1	1	1	OP7	0000 0000 1000 0000
1	1	0	0	0	OP8	0000 0001 0000 0000
1	1	0	0	1	OP9	0000 0010 0000 0000
1	1	0	1	0	OP10	0000 0100 0000 0000
1	1	0	1	1	OP11	0000 1000 0000 0000
1	1	1	0	0	OP12	0001 0000 0000 0000
1	1	1	0	1	OP13	0010 0000 0000 0000
1	1	1	1	0	OP14	0100 0000 0000 0000
1	1	1	1	1	OP15	1000 0000 0000 0000

Table 2: Truth Table For Component 4:16 Decoder Made From 2 3:8 Decoders

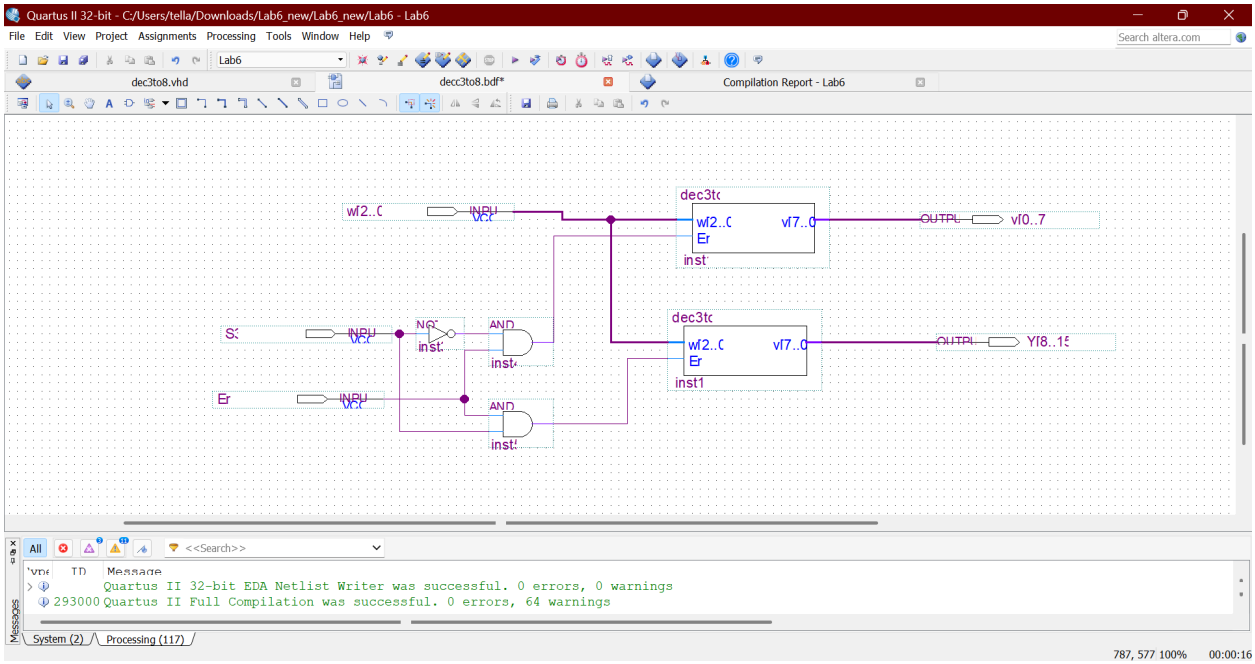


Figure 3: Block Schematic Diagram For 4:16 Decoder Using 2 3:8 Decoders, a NOT Gate, and Two AND Gates With Successful Compilation.

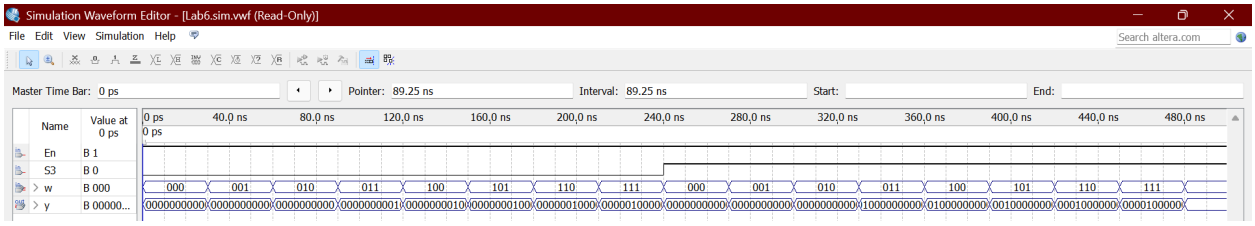


Figure 4: Waveform Output For 4:16 Decoders Made From 2 3:8 Decoders

FSM

The FSM cycles through nine states (s0 to s8), advancing to the next state on every rising clock edge and returning to state 0 when the Reset is active. The purpose of the FSM is to ensure the system runs in a fixed, repeating sequence so each ALU function is activated in order.

Clock	Reset	Present State (Dec)	Present State (S3 S2 S1 S0)	Next State (Dec)	Next State (NS3 NS2 NS1 NS0)	student_id[3:0] (501316925)
↑	1	X	XXXX	0	0000	0101 (5)
↑	0	0	0000	1	0001	0101 (5)
↑	0	1	0001	2	0010	0000 (0)
↑	0	2	0010	3	0011	0001 (1)
↑	0	3	0011	4	0100	0011 (3)
↑	0	4	0100	5	0101	0001 (1)
↑	0	5	0101	6	0110	0110 (6)
↑	0	6	0110	7	0111	1001 (9)
↑	0	7	0111	8	1000	0010 (2)
↑	0	8	1000	0	0000	0101 (5)
0	0	—	—	—	—	holds previous

Table 3: Truth Table For Component FSM Type Moore Following Student ID: 501316925. Modified so that states follow the order: 0→1→2→3→4→5→6→7→8→Back to 0.

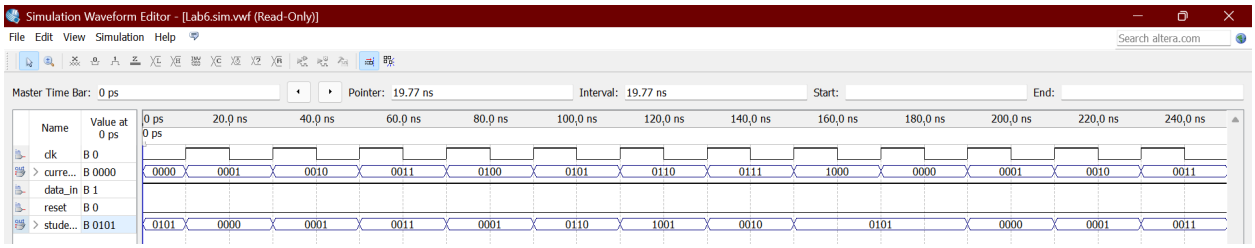


Figure 4: Waveform Output of FSM With States In Proper Sequence Corresponding to Digits From Student Number.

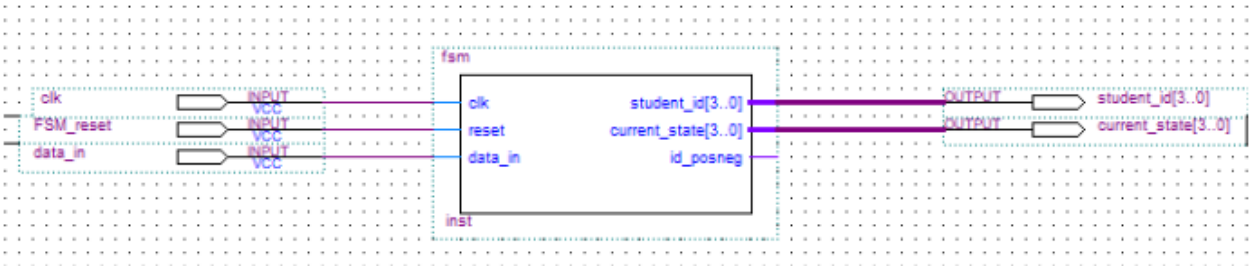


Figure 5: Block Diagram For FSM

Problem Set 1

The following schematic diagram stays consistent for all three problem sets. The individual components change per requirement of each problem set. The values of A and B for all problem sets were determined as follows:

Student ID: 501316925 —> A = $(69)_{16}$; B = $(25)_{16}$

Therefore, translating to A = $(0110\ 1001)_2$; B = $(0010\ 0101)_2$

ALU_1 performs the first set of arithmetic and logic operations required in the procedure, using the two 8-bit inputs stored in Latch 1 (A) and Latch 2 (B). Its purpose is to compute addition, subtraction, bitwise inversion, logical AND/OR, XOR, XNOR, NAND, and NOR based on the 16-bit microcode generated by the 4:16 decoder. Only one microcode line is active at a time, which allows the ALU to select the correct operation. The ALU processes the inputs on each rising clock edge and produces an 8-bit output, which is then split into two 4-bit values (R2 and R1) for the seven-segment display. The system shows the ALU results while following the sequential operation flow controlled by the FSM and decoder.

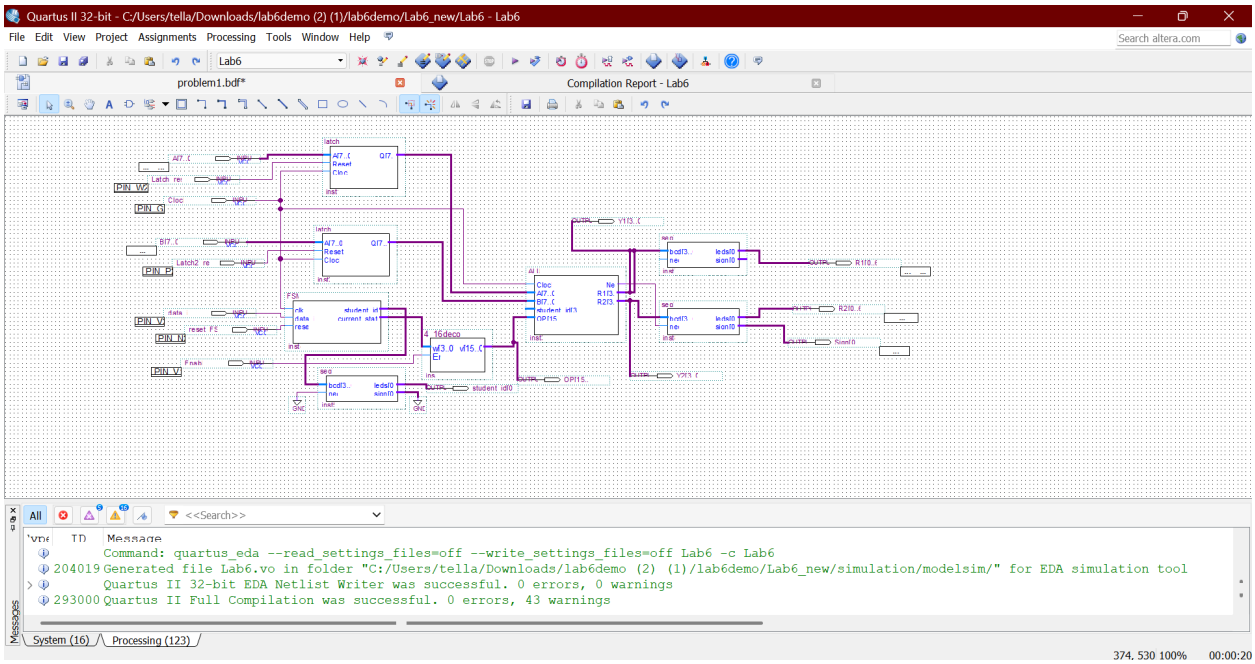


Figure 5: Block Schematic Diagram For Problem Set 1 With All Components With Successful Compilation

Purpose of Inputs and Outputs

Inputs

1. A[7..0] : First 8-bit operand stored in Latch 1, used for all ALU_1 operations.
2. B[7..0] : Second 8-bit operand stored in Latch 2, used together with A for arithmetic and logic functions.
3. Clock : Rising edge signal that updates the latches, FSM, and ALU computations.
4. Reset_A/Reset_B : Clears Latch 1 and Latch 2 to “00000000”, ensuring known starting values for A and B (sets values of Q and Q2 back to 0000000).
5. Reset_FSM : Returns the FSM to state 0 and sets the student_id[3..0] to the corresponding first digit.
6. Enable : Activates the registers so A and B can be set to their respective latches.
7. data_in : Used to change the state of the FSM when the Clock has a rising edge.

Outputs

1. R1[3..0] : Lower 4 bits of the ALU result, used for the least-significant HEX digit on the seven-segment display.
2. R2[3..0] : Upper 4 bits of the ALU result, used for the most-significant HEX digit on the seven-segment display.
3. Sign : Indicates if the ALU result is negative (MSB of the output).
4. student_id[3..0] : FSM output pattern used for the seven-segment display throughout problem 1.
5. current_state[3..0] : FSM state sent into decoder.

Clock	Data_in	Reset_FSM	Enable	w[3..0]	y[15..0]	Boolean Operation/Function
0	x	x	x	-	-	
1	x	1	x	x	x	
1	1	0	0	0	0	
1	1	0	1	0000	0000000000000001	sum(A, B)
1	1	0	1	0001	0000000000000010	diff(A, B)
1	1	0	1	0010	0000000000000100	$\bar{A}$
1	1	0	1	0011	0000000000001000	NOT($A \cdot B$)
1	1	0	1	0100	0000000000010000	NOT($A + B$)
1	1	0	1	0101	0000000000100000	$A \cdot B$
1	1	0	1	0110	0000000001000000	$A \oplus B$
1	1	0	1	0111	0000000010000000	$A + B$
1	1	0	1	1000	0000000100000000	NOT($A \oplus B$)

Table 4: Table of Microcode's Generated by Decoder For ALU_1

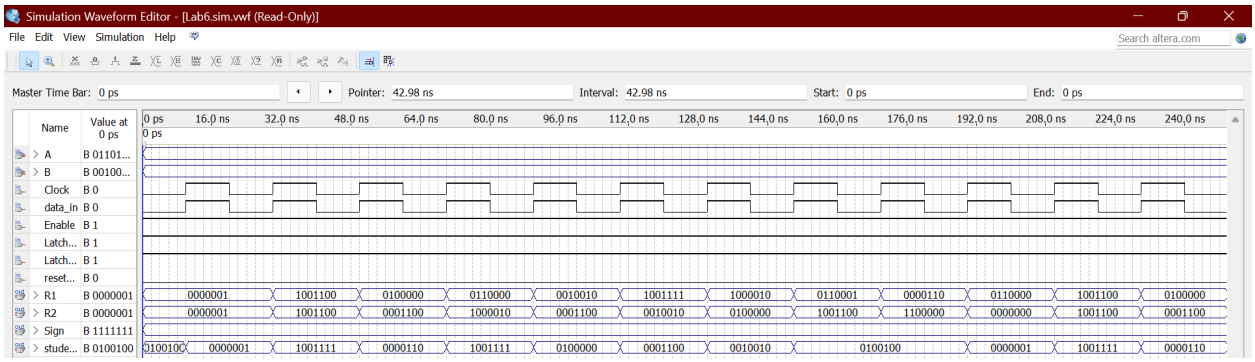


Figure 6: Waveform Output For Problem Set 1 Block Diagram

Problem Set 2

The block schematic diagram in problem set 2 is built with the same components as the previous problem, with the only difference being a modified ALU that implements the custom set of nine operations that were assigned. The interface of ALU_2 is the same with the 8-bit inputs A and B, 16-bit microcode OP, and 8-bit result. However, the internal case statement is expanded so each OP line now maps to the new functions from Modification F in the lab 6 manual. The FSM and 4:16 decoder still generate the microcodes based on the current state, so ALU_2 executes the nine operations in sequence as the states cycle from 0 to 0. The 8-bit result is again split into R2 and R1 (upper and lower 4 bits) for display on the seven-segment displays.

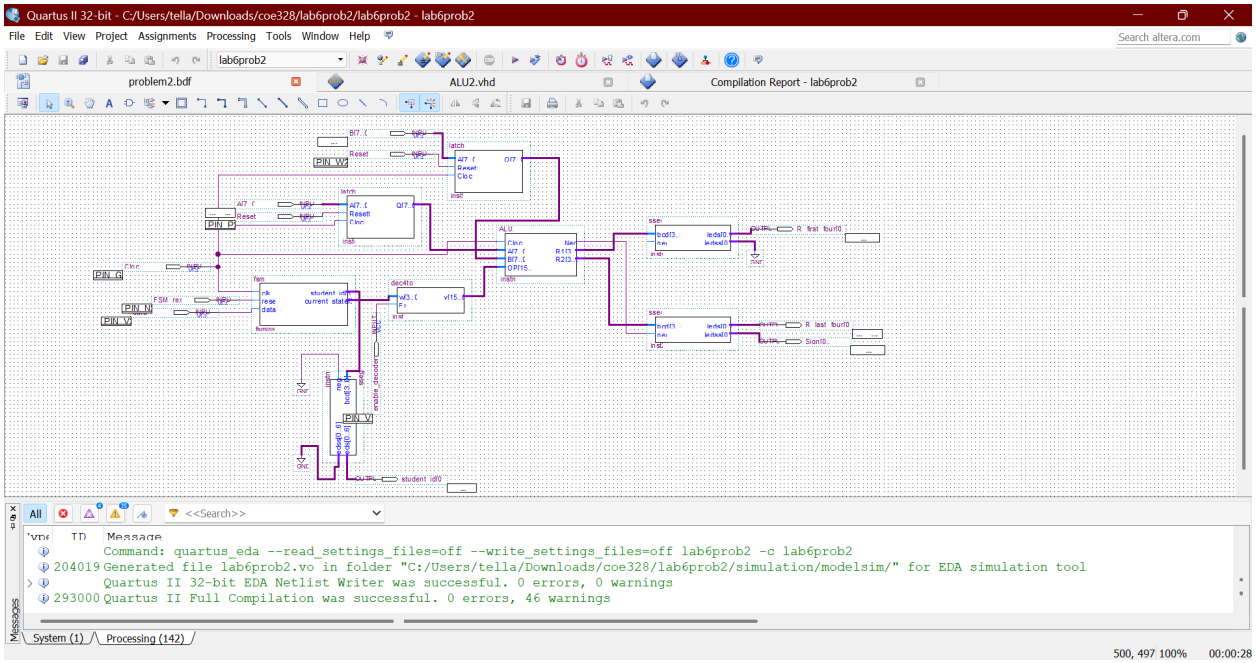


Figure 7: Block Schematic Diagram For Problem Set 2 With All Components (Modified ALU_2) With Successful Compilation

Purpose of Inputs and Outputs

The purpose of the inputs and outputs in problem set 2 remain the same as in problem set 1. The only changed component in problem set 2 is the modified ALU_2. The modified ALU_2 changes the values of the output, not the purposes of the inputs and outputs.

Clock	data_in	Reset_FSM	Enable	w[3..0]	y[15..0]	ALU2 Operations (SET A)
0	x	x	x	-	-	-
1	x	1	x	x	x	-
1	1	0	0	0	0	-
1	1	0	1	0000	0000000000000001	Decrement **B** by 9
1	1	0	1	0001	0000000000000010	Swap the lower and upper 4 bits of **B**
1	1	0	1	0010	0000000000000100	Shift **A** to left by 2 bits, input bit = 0 (SHL)
1	1	0	1	0011	0000000000001000	Produce the result of NANDing **A** and **B**
1	1	0	1	0100	0000000000010000	Find the greater value of **A** and **B** and produce the results (Max(**A**,**B**))
1	1	0	1	0101	0000000000100000	Invert the even bits of **B**
1	1	0	1	0110	0000000001000000	Produce null on the output
1	1	0	1	0111	0000000010000000	Replace the upper four bits of **B** by upper four bits of **A**
1	1	0	1	1000	0000000100000000	Show **A** on the output

Table 5: Table of Microcode's Generated by Decoder For ALU_2 (Modification F In Manual)

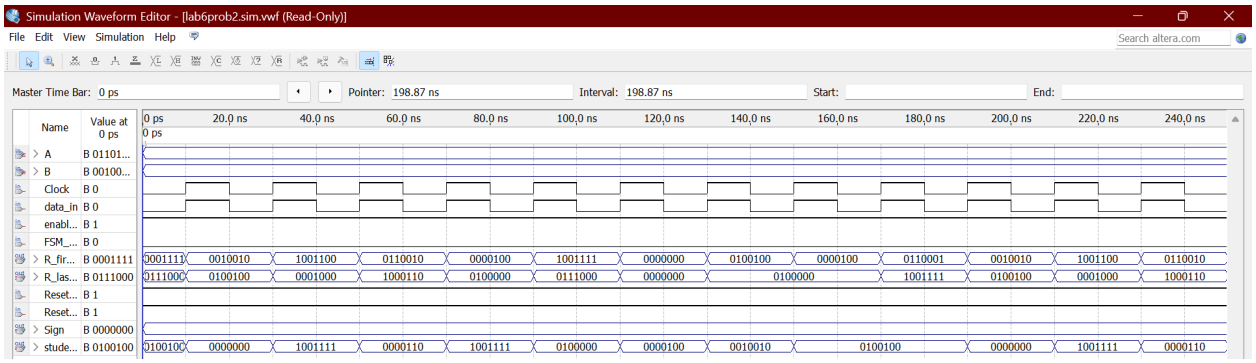


Figure 8: Waveform Output For Problem Set 2 Block Diagram With Modified ALU

Problem Set 3

The final problem set performs the comparison logic required for problem E that was assigned in the lab manual. This unit compares the two 4-bit digits of A with the current student_id value generated by the FSM. The ALU hardware remains the same as ALU_1. The difference comes from how the output is interpreted. A separate modified seven-segment display module (sseg_yn) was created to display either “Y” or “N” based on the comparison condition. The ALU_1 still produces the 8-bit output for each microcode instruction, but it ignores the numerical values and instead checks whether either digit of A is greater than the current student_id from the FSM. If the condition is true, the sseg_yn module lights the segments to show Y, and if it is false, it shows N. The FSM from problem set 1 was modified to incorporate a condition to check if either digit of A is greater than the current student ID.

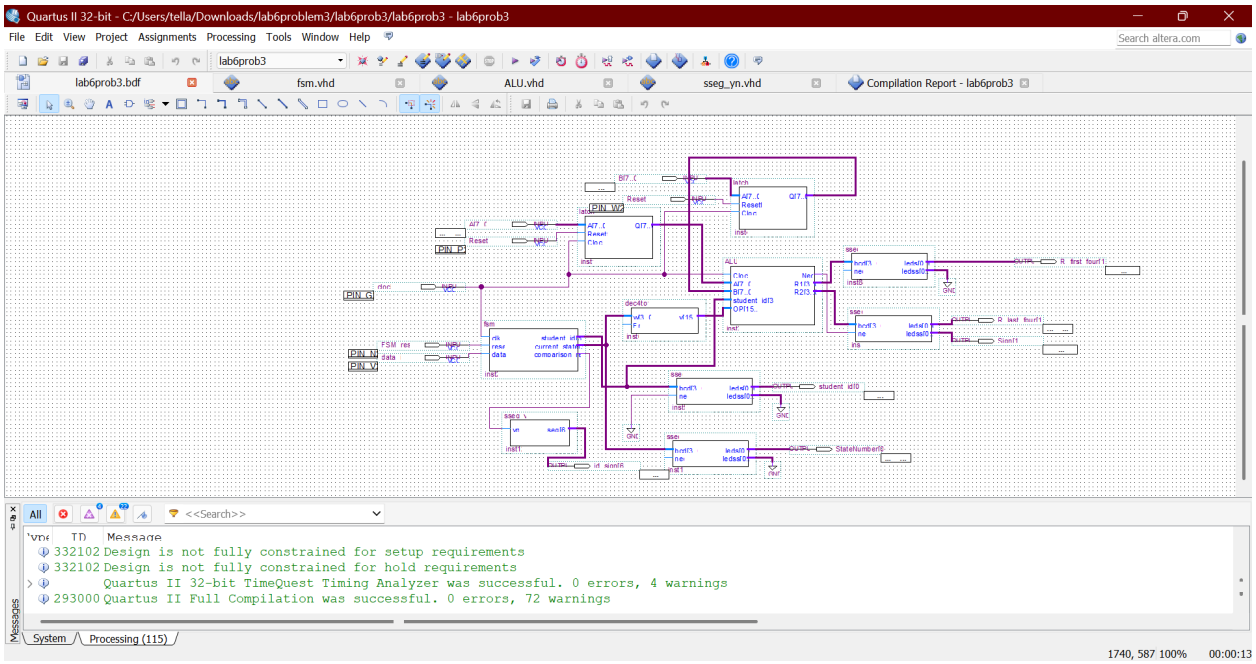


Figure 9: Block Schematic Diagram For Problem Set 3 With All Components (Modified FSM, Modified sseg_yn) With Successful Compilation

Purpose of Inputs and Outputs

Inputs

8. A[7..0] : First 8-bit value stored in Latch 1, used to compare its two digits against the FSM's student_id value.
9. B[7..0] : Second 8-bit value stored in Latch 2.
10. Clock : Rising edge signal that updates the latches, FSM, and refreshes the comparison logic.
11. Reset_A/Reset_B : Clears Latch 1 and Latch 2 to "00000000", ensuring known starting values for A and B.
12. Reset_FSM : Resets the FSM back to state 0.
13. Enable : Activates the registers so A and B can be set to their respective latches.
14. data_in : Used to change the state of the FSM when the Clock has a rising edge.

Outputs

6. R1[3..0] : Lower 4 bits of the ALU result, used for the least-significant HEX digit on the seven-segment display.
7. R2[3..0] : Upper 4 bits of the ALU result, used for the most-significant HEX digit on the seven-segment display.
8. Sign : Indicates if the ALU result is negative (MSB of the output).
9. student_id[3..0] : FSM output pattern used for the seven-segment display throughout problem 1.
10. id_sign[0..6] : Output from the modified sseg_yn module that displays 'Y' or 'N' depending on whether either digit of A is greater than the current student_id.

Clock	Data_in	Reset_FSM	Enable	w[3..0]	y[15..0]	Boolean Operation/Function
0	x	x	x	-	-	
1	x	1	x	x	x	
1	1	0	0	0	0	
1	1	0	1	0000	0000000000000001	sum(A, B)
1	1	0	1	0001	0000000000000010	diff(A, B)
1	1	0	1	0010	0000000000000100	$\bar{A}$
1	1	0	1	0011	0000000000001000	NOT($A \cdot B$)
1	1	0	1	0100	0000000000010000	NOT($A + B$)
1	1	0	1	0101	0000000000100000	$A \cdot B$
1	1	0	1	0110	0000000001000000	$A \oplus B$
1	1	0	1	0111	0000000010000000	$A + B$
1	1	0	1	1000	0000000100000000	NOT($A \oplus B$)

Table 6: Table of Microcode's Generated by Decoder For Problem Set 3 (Modification E in Manual) (ALU_1, Modified FSM)

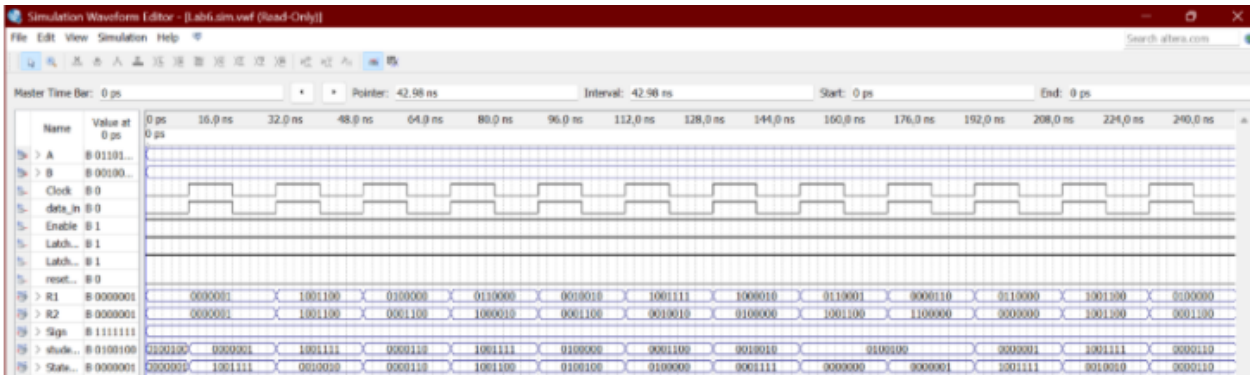


Figure 10: Waveform Output of Problem Set 3 Block Diagram

Conclusion

This lab incorporated latches, decoders, FSM control units, and different ALU designs to create a Simple General-Purpose Processor. The interaction between stored operands, microcode selection, and state-based outputs become clear through the implementation of each ALU, along with verification through waveform simulation. The final integration of the seven-segment display demonstrated how control logic and arithmetic outputs work together in a practical design. The results of problem 1 and 2 showcase the power of differing ALUs and how ALUs in general can be used to implement functions. The results of problem 3 show how FSMs can be altered to output specific results using comparison logic that are microcoded by the decoder and produced to the ALU. Overall, the lab reinforced how digital components combine to form functional systems. This strengthened the understanding of sequential logic and state-driven behaviour. The waveforms for all three problem sets are correct and confirm that the components work properly, as the output was as expected.

Current State	Student ID	Problem 1	Problem 2	Problem 3
0	5	8E	1C	1001100 (‘Y’)
1	0	44	52	1001100 (‘Y’)
2	1	96	A4	1001100 (‘Y’)
3	3	DE	DE	1001100 (‘Y’)
4	1	92	69	1001100 (‘Y’)
5	6	21	70	1001100 (‘Y’)
6	9	4C	00	0001001 (‘N’)
7	2	6D	65	1001100 (‘Y’)
8	5	B3	69	1001100 (‘Y’)

Table 7: Lab 6 Results Summary That Correctly Align With the Waveforms